

Nama : Elsa Marthalinda

NIM : 254107020204

Kelas : TI – 1G

LAPORAN JOBSHEET 14

14.2 Kegiatan Praktikum 1

14.2.1 Percobaan 1

- **Kode Program Mahasiswa07.java;**

```
package Pertemuan14;
public class Mahasiswa07 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa07() {
    }

    public Mahasiswa07(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampilInformasi() {
        System.out.println("NIM: " +this.nim  + " | " +
            "Nama: " +this.nama  + " | " +
            "Kelas: " +this.kelas + " | " +
            "IPK: " +this.ipk);
    }
}
```

- **Kode Program Node07.java;**

```
package Pertemuan14;
public class Node07 {
    Mahasiswa07 mahasiswa;
    Node07 left, right;

    public Node07() {
    }

    public Node07(Mahasiswa07 mahasiswa) {
        this.mahasiswa = mahasiswa;
    }
}
```

```

        left = right = null;
    }
}

```

➤ **Kode Program** BinaryTree07.java;

```

package Pertemuan14;
public class BinaryTree07 {
    Node07 root;

    public BinaryTree07() {
        root = null;
    }

    public boolean isEmpty() {
        return root == null;
    }

    public void add(Mahasiswa07 mahasiswa) {
        Node07 newNode = new Node07(mahasiswa);
        if (isEmpty()) {
            root = newNode;
        } else {
            Node07 current = root;
            Node07 parent = null;
            while(true) {
                parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;
                    if (current == null) {
                        parent.left = newNode;
                        return;
                    }
                } else {
                    current = current.right;
                    if (current == null) {
                        parent.right = newNode;
                        return;
                    }
                }
            }
        }
    }

    boolean find(double ipk) {
        boolean result = false;
        Node07 current = root;
        while (current != null) {
            if (current.mahasiswa.ipk == ipk) {
                result = true;
                break;
            }
        }
    }
}

```

```

        } else if (ipk > current.mahasiswa.ipk) {
            current = current.right;
        } else {
            current = current.left;
        }
    }
    return result;
}

void traversePreOrder(Node07 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

void traverseInOrder(Node07 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

void traversePostOrder(Node07 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

Node07 getSuccessor(Node07 del) {
    Node07 successor = del.right;
    Node07 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
    }
}

```

```

        return;
    }
    //cari node (current) yang akan dihapus
    Node07 parent = root;
    Node07 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if(current.mahasiswa.ipk == ipk) {
            break;
        } else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        } else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }
    //penghapusan
    if (current == null) {
        System.out.println("Data tidak ditemukan");
        return;
    } else {
        //jika tidak ada anak (leaf), maka node dihapus
        if (current.left == null && current.right == null) {
            if (current == root) {
                root = null;
            } else {
                if (isLeftChild) {
                    parent.left = null;
                } else {
                    parent.right = null;
                }
            }
        }
        } else if (current.left == null) { //jika hanya punya 1 anak (kanan)
            if (current == root) {
                root = current.right;
            } else {
                if (isLeftChild) {
                    parent.left = current.right;
                } else {
                    parent.right = current.right;
                }
            }
        }
        } else if (current.right == null) { //jika hanya punya 1 anak (kiri)
            if (current == root) {
                root = current.left;
            } else {
                if (isLeftChild) {

```

```

        parent.left = current.left;
    } else {
        parent.right = current.left;
    }
}
} else { //jika punya 2 anak
    Node07 successor = getSuccessor(current);
    System.out.println("Jika 2 anak, current = ");
    successor.mahasiswa.tampilInformasi();
    if (current == root) {
        root = successor;
    } else {
        if(isLeftChild) {
            parent.left = successor;
        } else {
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}
}
}
}

```

➤ **Kode Program** BinaryTreeMain07.java;

```

package Pertemuan14;

public class BinaryTreeMain07 {
    public static void main(String[] args) {
        BinaryTree07 bst = new BinaryTree07();

        bst.add(new Mahasiswa07("244160121", "Ali", "A", 3.57));
        bst.add(new Mahasiswa07("244160221", "Badar", "B", 3.85));
        bst.add(new Mahasiswa07("244160185", "Candra", "C", 3.21));
        bst.add(new Mahasiswa07("244160220", "Dewi", "B", 3.54));

        System.out.println("\nDaftar semua mahasiswa (in oder traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data mahasiswa:");
        System.out.print("Cari mahasiswa dengan ipk: 3.54 : ");
        String hasilCari = bst.find(3.54) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        System.out.print("Cari mahasiswa dengan ipk: 3.22 : ");
        hasilCari = bst.find(3.22) ? "Ditemukan" : "Tidak ditemukan";
        System.out.println(hasilCari);

        bst.add(new Mahasiswa07("244160131", "Devi", "A", 3.72));
        bst.add(new Mahasiswa07("244160205", "Ehsan", "D", 3.37));
        bst.add(new Mahasiswa07("244160170", "Fizi", "B", 3.46));
    }
}

```

```

        System.out.println("\nDaftar semua mahasiswa setelah penambahan 3
mahasiswa:");
        System.out.println("InOrder Traversal:");
        bst.traverseInOrder(bst.root);
        System.out.println("\nPreOrder Traversal:");
        bst.traversePreOrder(bst.root);
        System.out.println("\nPostOrder Traversal:");
        bst.traversePostOrder(bst.root);

        System.out.println("\nPenghapusan data mahasiswa");
        bst.delete(3.57);
        System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1
mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);
    }
}

```

➤ Hasil Output

Daftar semua mahasiswa (in oder traversal):
 NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
 NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
 NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
 NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Pencarian data mahasiswa:
 Cari mahasiswa dengan ipk: 3.54 : Ditemukan
 Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:
 InOrder Traversal:
 NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
 NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
 NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
 NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
 NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
 NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
 NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

PreOrder Traversal:
 NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
 NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
 NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
 NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
 NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
 NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85
 NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

PostOrder Traversal:
 NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
 NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85
NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

Penghapusan data mahasiswa

Jika 2 anak, current =

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

14.2.2 Pertanyaan Percobaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?
 - BST lebih efektif karena setiap node memiliki aturan $\text{left} < \text{parent} < \text{right}$, sehingga pencarian bisa langsung memilih arah kiri/kanan tanpa menelusuri semua node. Kompleksitas $O(\log n)$ vs $O(n)$ pada binary tree biasa.
2. Untuk apakah di class **Node**, kegunaan dari atribut **left** dan **right**?
 - **left**: Digunakan untuk menyimpan alamat/referensi ke anak kandung sebelah kiri (left child), yaitu node yang memiliki nilai data (dalam kasus ini nilai IPK) lebih kecil dari node saat ini.
 - **right**: Digunakan untuk menyimpan alamat/referensi ke anak kandung sebelah kanan (right child), yaitu node yang memiliki nilai data (IPK) yang lebih besar atau sama dengan node saat ini
3. a. Untuk apakah kegunaan dari atribut root di dalam class **BinaryTree**?
 - root adalah entry point ke seluruh tree — semua operasi (add, find, delete, traversal) dimulai dari root.b. Ketika objek tree pertama kali dibuat, apakah nilai dari **root**?
 - Ketika objek tree pertama kali dibuat, nilai root adalah null.
4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

- Jika tree kosong, isEmpty() mengembalikan true, maka node baru langsung dijadikan root = newNode.
5. Perhatikan method **add()**, di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detil untuk apa baris program tersebut?

```
parent = current;
if (mahasiswa.ipk < current.mahasiswa.ipk) {
    current = current.left;
    if (current == null) {
        parent.left = newNode;
        return;
    }
} else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}
```

- Baris parent = current menyimpan posisi parent sebelum bergerak ke child. Perbandingan IPK menentukan arah kiri/kanan. Jika current == null ditemukan, itulah posisi sisipan dan parent.left/right = newNode.
6. Jelaskan langkah-langkah pada method **delete()** saat menghapus sebuah node yang memiliki dua anak. Bagaimana method **getSuccessor()** membantu dalam proses ini?
- **Langkah penghapusan:** Method mencari node pengganti menggunakan getSuccessor() , menyambungkan parent dari node yang dihapus ke node pengganti tersebut , lalu memindahkan silsilah anak kiri node yang dihapus ke node pengganti (successor.left = current.left)
 - **Peran getSuccessor():** Membantu mencari **nilai terkecil dari sub-tree sebelah kanan** node yang akan dihapus . Node ini adalah pengganti yang paling aman agar aturan/sifat struktur *Binary Search Tree* tidak rusak setelah ada data yang dihapus.

14.3 Kegiatan Praktikum 2

- **Kode Program** BinaryTreeArray07.java;
package Pertemuan14;
public class BinaryTreeArray07 {
Mahasiswa07[] dataMahasiswa;
int idxLast;

public BinaryTreeArray07() {
this.dataMahasiswa = new Mahasiswa07[10];
}

```

void populateData(Mahasiswa07 dataMhs[], int idxLast) {
    this.dataMahasiswa = dataMhs;
    this.idxLast = idxLast;
}

void traverseInOrder(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswa[idxStart] != null) {
            traverseInOrder(2 * idxStart + 1);
            dataMahasiswa[idxStart].tampilInformasi();
            traverseInOrder(2 * idxStart + 2);
        }
    }
}
}
}

```

➤ **Kode Program** BinaryTreeArrayMain07.java;

```

package Pertemuan14;
public class BinaryTreeArrayMain07 {
    public static void main(String[] args) {

        BinaryTreeArray07 bta = new BinaryTreeArray07();
        Mahasiswa07 mhs1 = new Mahasiswa07("244160121", "Ali", "A", 3.57);
        Mahasiswa07 mhs2 = new Mahasiswa07("244160185", "Candra", "C", 3.41);
        Mahasiswa07 mhs3 = new Mahasiswa07("244160221", "Badar", "B", 3.75);
        Mahasiswa07 mhs4 = new Mahasiswa07("244160220", "Dewi", "B", 3.35);

        Mahasiswa07 mhs5 = new Mahasiswa07("244160131", "Devi", "A", 3.48);
        Mahasiswa07 mhs6 = new Mahasiswa07("244160205", "Ehsan", "D", 3.61);
        Mahasiswa07 mhs7 = new Mahasiswa07("244160170", "Fajar", "B", 3.86);

        Mahasiswa07[] dataMahasiswa = {mhs1, mhs2, mhs3, mhs4, mhs5, mhs6, mhs7,
null, null, null};
        int idxlast = 6;
        bta.populateData(dataMahasiswa, idxlast);
        System.out.println("\nInorder Traverse Mahasiswa:");
        bta.traverseInOrder(0);
    }
}

```

➤ **Hasil Output**

Inorder Traverse Mahasiswa:			
NIM: 244160220	Nama: Dewi	Kelas: B	IPK: 3.35
NIM: 244160185	Nama: Candra	Kelas: C	IPK: 3.41
NIM: 244160131	Nama: Devi	Kelas: A	IPK: 3.48
NIM: 244160121	Nama: Ali	Kelas: A	IPK: 3.57
NIM: 244160205	Nama: Ehsan	Kelas: D	IPK: 3.61
NIM: 244160221	Nama: Badar	Kelas: B	IPK: 3.75
NIM: 244160170	Nama: Fajar	Kelas: B	IPK: 3.86

14.3.2 Pertanyaan Percobaan

1. Apakah kegunaan dari atribut `data` dan `idxLast` yang ada di class **BinaryTreeArray**?
 - `data` digunakan untuk menyimpan seluruh node binary tree dalam bentuk array. Setiap indeks array merepresentasikan satu node pada tree.
 - `idxLast` digunakan untuk menyimpan indeks terakhir yang terisi data pada array, sehingga program mengetahui batas akhir data yang valid di dalam binary tree.
2. Apakah kegunaan dari method **populateData()**?
 - Method `populateData()` digunakan untuk mengisi array binary tree dengan data mahasiswa sekaligus mengatur nilai `idxLast`. Method ini mempermudah proses memasukkan data tree ke dalam array sebelum dilakukan traversal.
3. Apakah kegunaan dari method **traverseInOrder()**?
 - Digunakan untuk menampilkan isi binary tree dengan urutan traversal **InOrder**, yaitu: Mengunjungi subtree kiri, menampilkan node/root, mengunjungi subtree kanan .
 - Pada binary search tree, traversal inorder menghasilkan data yang terurut dari nilai terkecil ke terbesar.
4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?
 - $\text{Left child} = 2 * \text{indeks} + 1$
 $\text{Right child} = 2 * \text{indeks} + 2$
Jika node berada pada indeks 2, maka:
 $\text{Left child} = 2 * 2 + 1 = 5$
 $\text{Right child} = 2 * 2 + 2 = 6$
Jadi: Left child berada di indeks **5**, Right child berada di indeks **6**
5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?
 - Statement `int idxLast = 6` digunakan untuk menandai bahwa indeks terakhir yang berisi data pada array adalah indeks ke-6. Karena array diisi 7 data mahasiswa (indeks 0–6), maka traversal hanya akan memproses data sampai indeks tersebut dan tidak membaca elemen kosong setelahnya.
6. Mengapa indeks $2 * \text{idxStart} + 1$ dan $2 * \text{idxStart} + 2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

- Karena binary tree disimpan dalam bentuk array, hubungan parent dan child ditentukan berdasarkan rumus indeks
 Anak kiri node ke-i berada di indeks $2i+1$
 Anak kanan node ke-i berada di indeks $2i+2$

14.4 Tugas Praktikum

- **Modifikasi Kode Program** BinaryTree07.java;

//penambahan rekursif (no 1)

```
public void addRekursif(Mahasiswa07 mahasiswa) {
    root = addRekursif(root, mahasiswa);
}
private Node07 addRekursif(Node07 current, Mahasiswa07 mahasiswa) {
    if (current == null) {
        return new Node07(mahasiswa);
    }
    if (mahasiswa.ipk < current.mahasiswa.ipk) {
        current.left = addRekursif(current.left, mahasiswa);
    } else {
        current.right = addRekursif(current.right, mahasiswa);
    }
    return current;
}
```

//penambahan cari min IPK (no 2)

```
public void cariMinIPK() {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    Node07 current = root;
    while (current.left != null) {
        current = current.left;
    }
    current.mahasiswa.tampilInformasi();
}
```

//penambahan cari max IPK (no 2)

```
public void cariMaxIPK() {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    Node07 current = root;
```

```

        while (current.right != null) {
            current = current.right;
        }
        current.mahasiswa.tampilInformasi();
    }

    //penambahan tampil mhs ipk di atas batas (no 3)
    public void tampilMahasiswaIPKdiAtas(double ipkBatas) {
        tampilMahasiswaIPKdiAtasHelper(root, ipkBatas);
    }
    private void tampilMahasiswaIPKdiAtasHelper(Node07 current, double
ipkBatas) {
        if (current != null) {
            tampilMahasiswaIPKdiAtasHelper(current.left, ipkBatas);
            if (current.mahasiswa.ipk > ipkBatas) {
                current.mahasiswa.tampilInformasi();
            }
            tampilMahasiswaIPKdiAtasHelper(current.right, ipkBatas);
        }
    }
}

```

➤ Hasil Output

Daftar semua mahasiswa (in oder traversal):

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Pencarian data mahasiswa:

Cari mahasiswa dengan ipk: 3.54 : Ditemukan

Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:

InOrder Traversal:

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

PreOrder Traversal:

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

PostOrder Traversal:

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85
NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

Penghapusan data mahasiswa

Jika 2 anak, current =

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Mahasiswa dengan IPK terkecil:

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

Mahasiswa dengan IPK terbesar:

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Mahasiswa dengan IPK di atas 3.50:

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

➤ **Modifikasi Kode Program** BinaryTreeArray07.java;

```
void add(Mahasiswa07 data) {
    if (idxLast < dataMahasiswa.length - 1) {
        idxLast++;
        dataMahasiswa[idxLast] = data;
    } else {
        System.out.println("Array penuh");
    }
}

void traversePreOrder(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswa[idxStart] != null) {
            dataMahasiswa[idxStart].tampilInformasi();
        }
    }
}
```

```
        traversePreOrder(2 * idxStart + 1);
        traversePreOrder(2 * idxStart + 2);
    }
}
```

➤ **Hasil Output**

Inorder Traverse Mahasiswa:

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.35

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.41

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.48

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.61

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.75

NIM: 244160170 | Nama: Fajar | Kelas: B | IPK: 3.86

Preorder Traverse Mahasiswa:

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.41

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.35

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.48

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.75

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.61

NIM: 244160170 | Nama: Fajar | Kelas: B | IPK: 3.86